Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai-25

LONG-TERM MEMORY ENGINE FOR AI AGENTS

Third Project Review

GUIDE:

DR. BAMA SRINIVASAN

DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

CEG, ANNA UNIVERSITY

TEAM MEMBERS:

M. KRITHIKRAJ, 2023115004

ALSTON S, 2023115006

ARYAN I, 2023115021

HARI SRINIVASAN, 2023115022

PROBLEM STATEMENT AND OBJECTIVES

- An AI agent starts every session with a blank memory. Instructions and decisions from earlier sessions are lost unless the user repeats them.
- Pasting the whole history into the prompt is slow and costly, and models miss details in the middle of long inputs [1].
- When an instruction changes, the agent cannot tell the new value from the old one, so it may act on the old one.

Objectives

1. Retain useful information for AI agents across sessions.
2. Keep the updated instruction while preserving the earlier versions.
3. Link every stored assertion back to the original message so it can be verified.
4. Fine-tune a small local model to extract assertions, combine them and answer with citations.
5. Retrieve relevant information quickly within a limited context size.
6. Evaluate the engine with repeatable tests on a public benchmark, LoCoMo [2].

LITERATURE SURVEY

Work	How it works	Reported result	Limitation
MemGPT [3], 2023	The model moves data between the prompt and storage by function calls	92.5% vs 32.1% for GPT-4 with a summary	A missed function call loses the update
Zep / Graphiti [4], 2025	Temporal graph; a contradiction closes the old fact	71.2% vs 60.2% for the full history on LongMemEval [5]	Single run; graph database is the main store
Mem0 [6], 2025	An LLM picks add, update, delete or no-op for each fact	66.9 vs 72.9 for the full history on LoCoMo; p95 1.4 s vs 17 s	One benchmark; open-source release dropped update and delete
Supermemory [7], 2026	Small memories linked to source chunks, with two timestamps	Vendor report only	Closed source, not peer reviewed
Hindsight [8], 2025	Retain, recall, reflect; fused recall in a token budget	Authors' own runs	Not peer reviewed; budget not stated

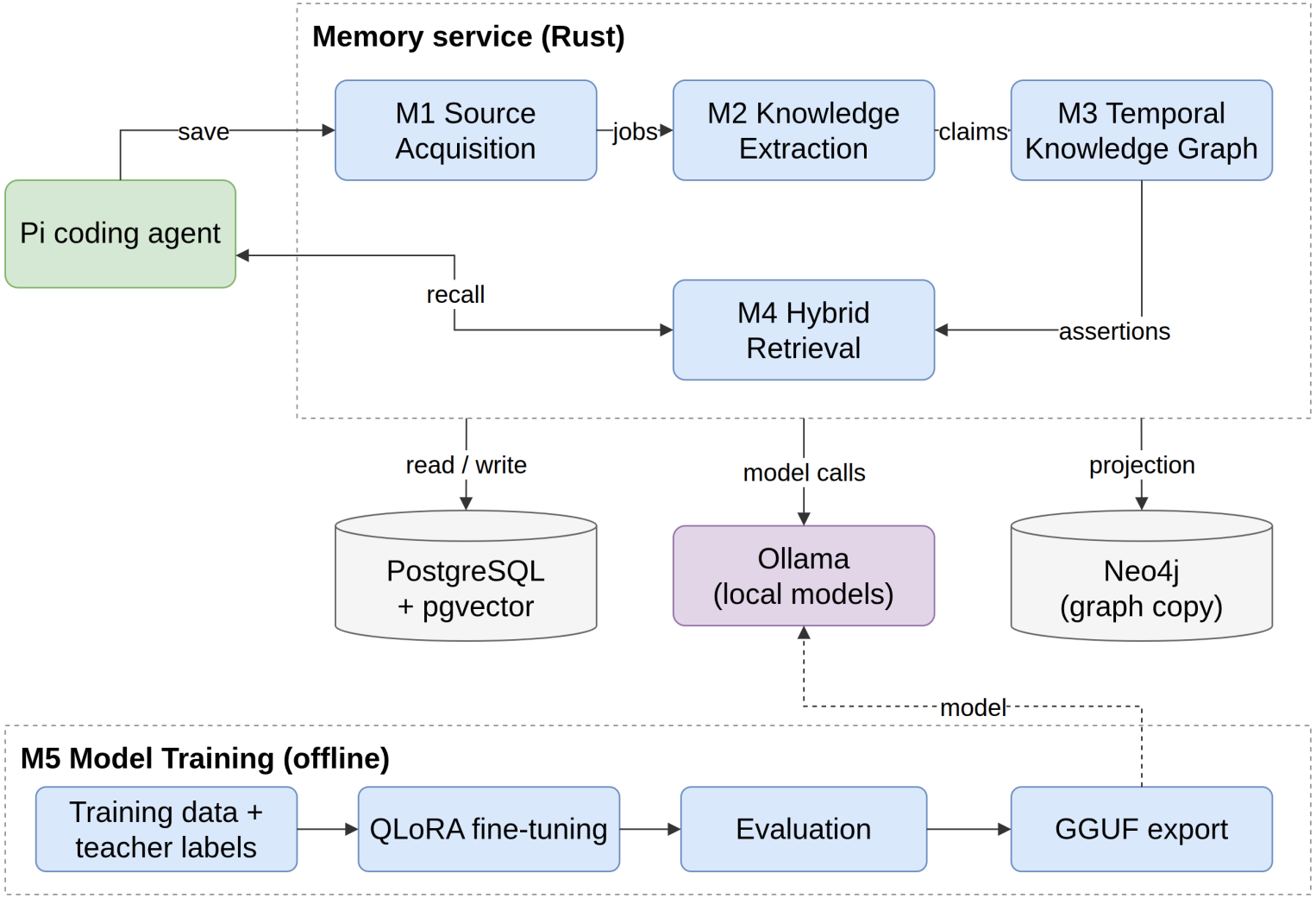
All numbers are as reported by the authors on their own setups. We did not rerun these systems.

GAPS IN EXISTING WORK AND OUR APPROACH

Gap	Seen in	How we address it
G1. The model decides how memory changes, so one wrong call loses data	MemGPT, Mem0	Fixed rules in code decide every update
G2. Changed values are overwritten or deleted	Mem0, MemGPT	Every change is a new version with the dates it was valid
G3. A memory is hard to check against what the user said	MemGPT, Mem0, Supermemory	Every assertion carries an exact quote from the source message
G4. Extraction depends on large hosted models	Zep, Mem0	A fine-tuned 4B model runs on our own laptop
G5. Results are hard to compare across papers	All five	Conditions fixed before the run, two judges, one-command rerun

The architecture on the next slide is built around these five choices.

SYSTEM ARCHITECTURE



Input: messages and tool results from the Pi coding agent [9].

M1 saves them in PostgreSQL and queues a job.

M2 extracts assertions with quotes, using the local model in Ollama.

M3 stores every version and copies it to Neo4j.

M4 searches both stores and packs the result into a token budget.

M5 trains the model offline.

Output: recalled memories with their quotes, sent back to Pi.

M1 to M4 run as one Rust service. Tokio workers run M2 and M3 in the background.

RUNNING EXAMPLE: MAYA

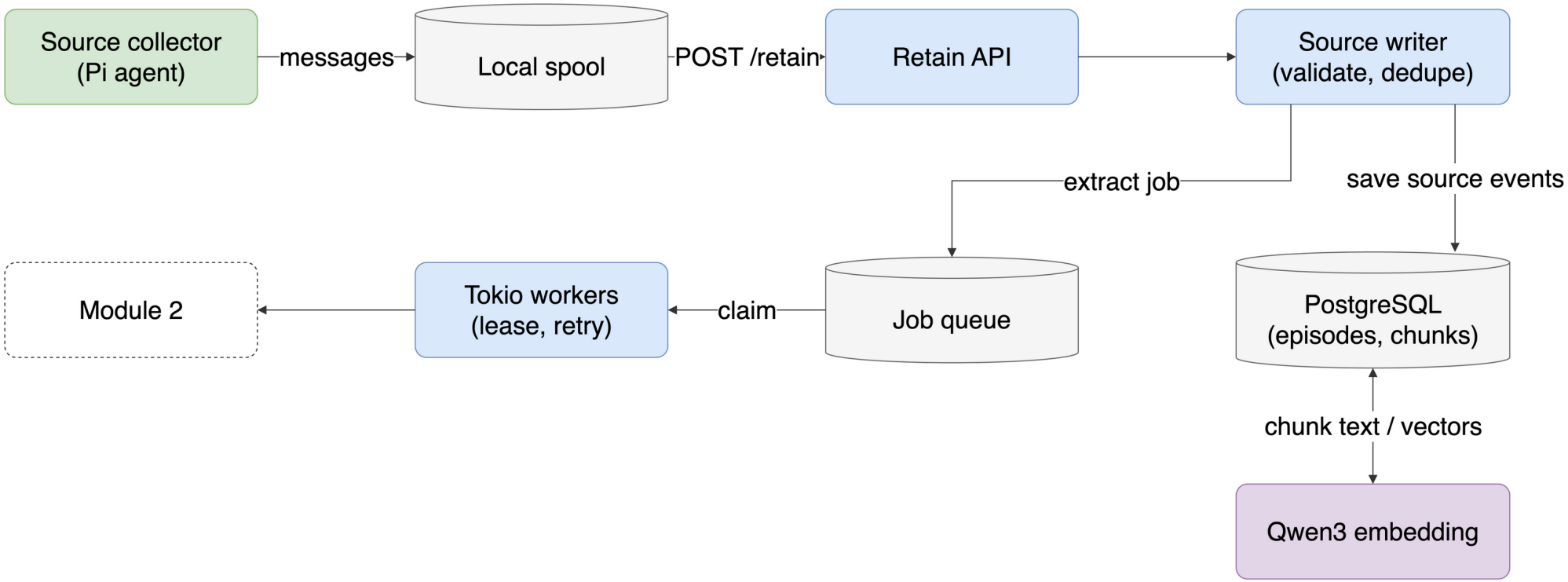
Event	Session, time	Message
E1	S1, user, 1 Oct	My name is Maya. I live in Chennai. I work as a software engineer at NovaTech, and I work remotely.
E2	S1, assistant	Nice to meet you, Maya.
E3	S2, user, 10 Oct	I have moved to Bengaluru. Bengaluru is now my home city.
E4	S2, assistant	Noted. You now live in Bengaluru.
E5	S3, user, 13 Oct	Where do I live now, where did I live before, and where do I work?

Goal for E5: answer Bengaluru (since 10 Oct), Chennai before that, and NovaTech, each with its source message.

An agent with no memory cannot answer. One that kept only E1 would wrongly say Chennai.

The next five slides follow this example through M1 to M5.

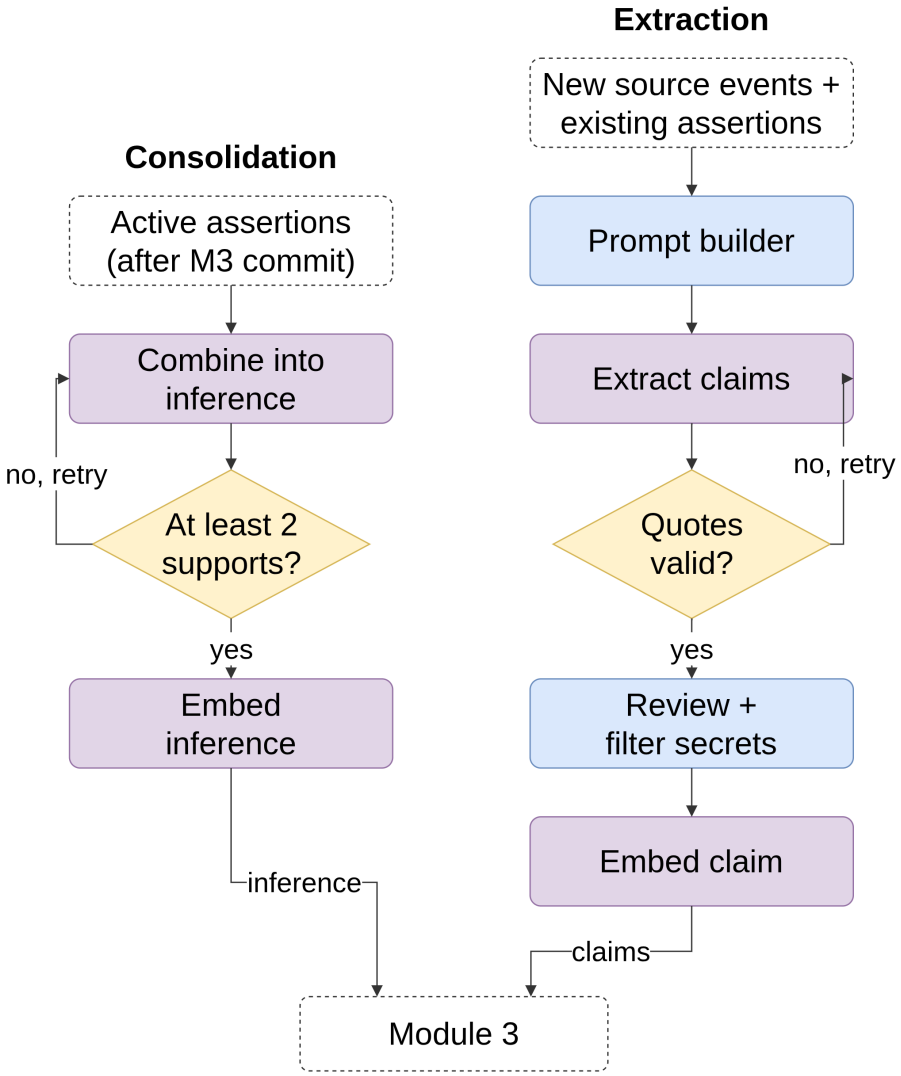
MODULE 1: SOURCE ACQUISITION AND PROCESSING



- Pi writes each message to a local spool first, so nothing is lost if the service is down.
- The retain API stores the raw events, splits them into chunks and adds a keyword index [10] and a 1,024-number vector [11].
- A repeated batch with the same idempotency key is ignored.
- It queues an extract job. Workers retry a failed job up to 5 times.

Maya: E3 and E4 become episode EP2, chunk C3 and job extract:EP2.

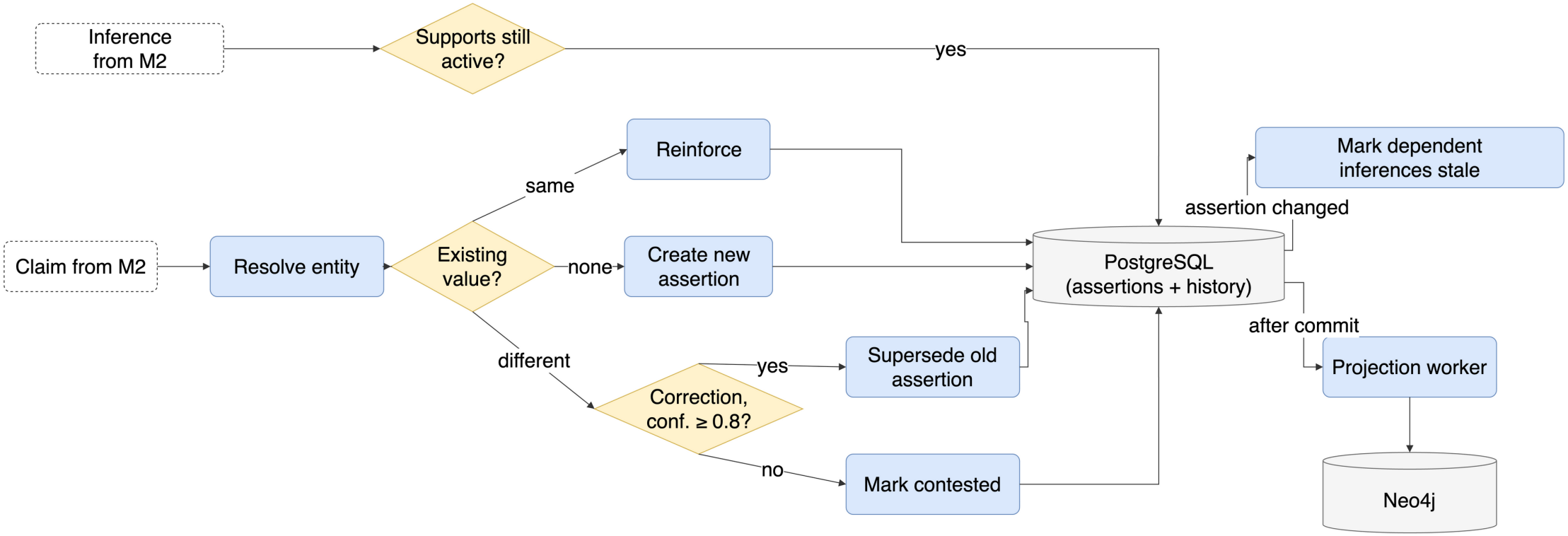
MODULE 2: KNOWLEDGE EXTRACTION AND CONSOLIDATION



- The fine-tuned model turns new events into claims, each with an exact quote.
- Bad output goes back to the model for repair, at most twice. Claims that look like passwords or keys are dropped.
- A review step checks that a correction is really stated.
- Consolidation combines two or more assertions into an inference.

Maya: E3 gives “home city Bengaluru”, correction true, confidence 0.95.

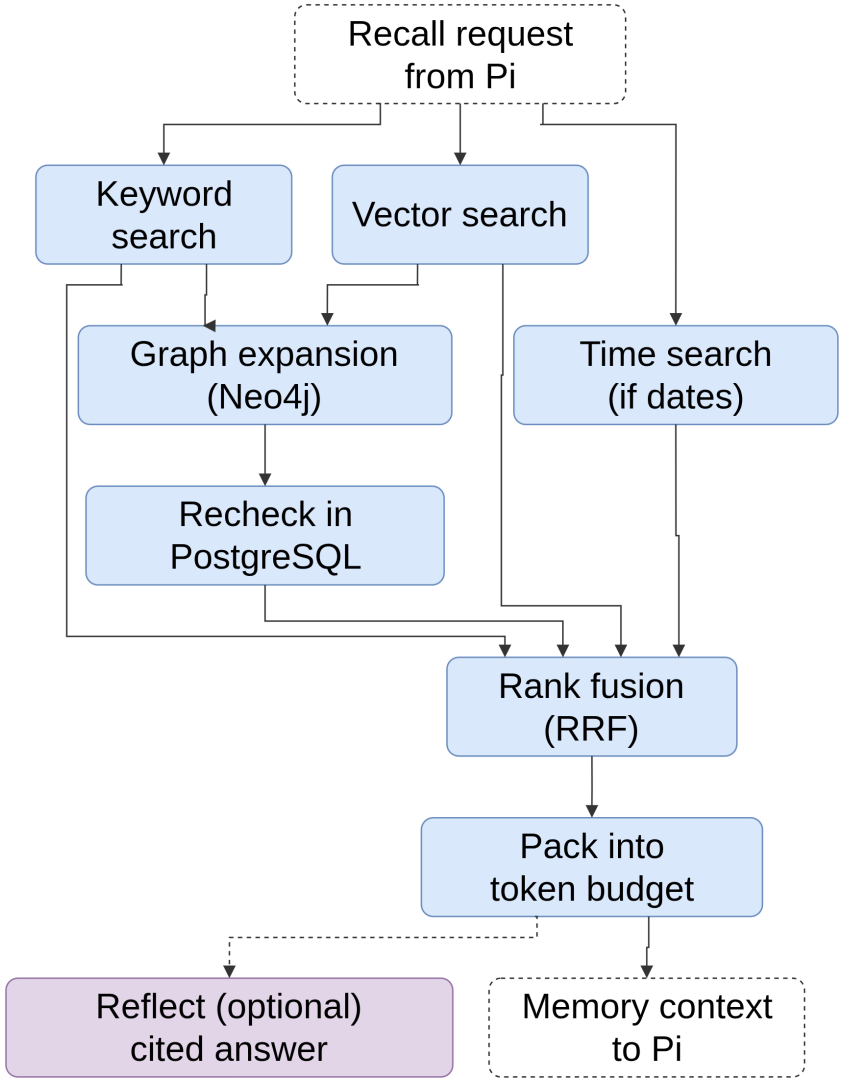
MODULE 3: TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT



- The only module that writes assertions.
- A new value supersedes the old one only if it is a correction with confidence of at least 0.8. Otherwise both are marked contested.
- Old values keep their valid dates. Inferences built on a changed assertion become stale.
- After commit, a worker copies changes to Neo4j [12].

Maya: R2 Bengaluru supersedes R1 Chennai, which stays valid from 1 to 10 Oct.

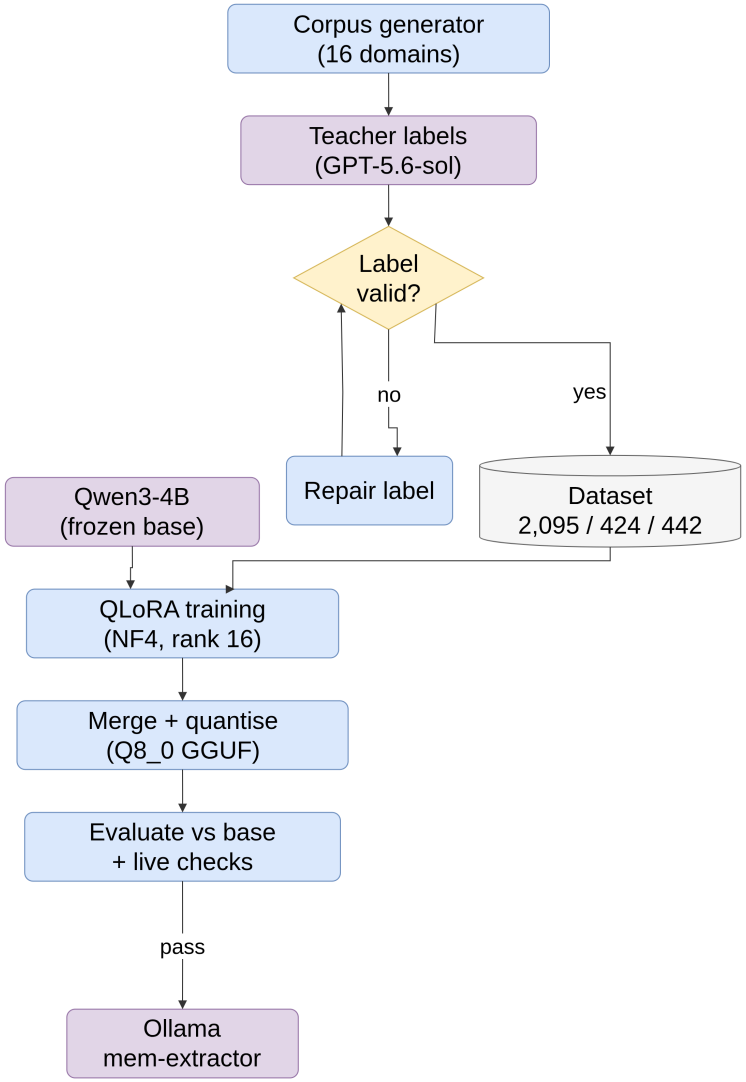
MODULE 4: HYBRID RETRIEVAL AND GROUNDED REASONING



- Keyword, vector, date and graph search run together.
- Graph hits are rechecked in PostgreSQL.
- Reciprocal Rank Fusion [13] merges the lists. The best results are packed into the token budget with their quotes.
- An optional step writes an answer that cites only retrieved IDs.

Maya: for E5, R2 (Bengaluru) and the E1 text (Chennai, NovaTech) come back together.

MODULE 5: MODEL TRAINING AND EVALUATION



- Runs offline. GPT-5.6-sol writes and labels synthetic conversations in 16 domains.
- Qwen3-4B is fine-tuned with QLoRA [14], exported to GGUF and served by Ollama.
- A new version is used only if it scores at least as well as the old one on the dev conversations.

Maya: a training row pairs a conversation like E1 to E4 with the expected claims and quotes.

ALGORITHMS (1/3)

Algorithm 4.1 Priority-based context selection (M2)

Input: existing assertions, new messages, input limit

IF all existing assertions fit:

    RETURN all of them

Score each assertion: +3 if its subject appears in the new messages, +2 if its value does

Sort by score, newest first on ties

Find the longest sorted prefix that fits, by binary search

RETURN that prefix in its original order

Algorithm 4.2 Consolidation and invalidation (M2, M3)

Input: active assertions about the changed subjects

Ask the model to combine related assertions into inferences

Keep an inference only if it has two or more supports

Check that every support is still active, then save with links

WHEN a support changes or is retracted:

    find inferences that depend on it

    mark them stale

ALGORITHMS (2/3)

Algorithm 4.3 Entity resolution and versioning (M3)

Input: a new assertion and the stored memory

Match the subject by type and name, or a unique alias; otherwise create it

IF the same assertion exists:

    attach the new quote

ELSE IF the slot has no conflicting value:

    store it as active

ELSE IF correction AND confidence ≥ 0.8:

    supersede the old value, keep it as history

ELSE:

    mark both values contested

Algorithm 4.4 Hybrid retrieval and graph traversal (M4)

Input: a question and an optional date range

Run keyword search [10] and vector search [11]

IF there is a date range:

    add assertions valid in that range

Follow graph links from the top hits for up to two rounds

Recheck graph hits in PostgreSQL

Drop results outside their validity

RETURN the candidate lists

ALGORITHMS (3/3)

Algorithm 4.5 Rank fusion and context packing (M4)

Input: ranked candidate lists and a token budget

FOR each memory:

    score = Σ 1 / (60 + rank) over all lists [13]

Sort by score, then by ID

FOR each memory in order:

    build a block with the assertion and its quotes

    add it if it fits the budget

RETURN the packed context

Algorithm 4.6 Source-grounded answering (M4)

Input: a question and the retrieved memories

IF nothing was retrieved:

    RETURN “Insufficient evidence.”

Ask the model to answer only from the retrieved memories

Check the output format and every cited ID

IF it answers without citations:

    reject it

RETURN the answer with its citations

TOOLS AND TECHNOLOGIES

Tool	Role
Rust, Tokio, Axum, SQLx	Memory service: API, background workers, database access
PostgreSQL 17 + pgvector [10], [11]	Source of truth; keyword (GIN) and vector (HNSW) search
Neo4j 5.26 [12]	Graph copy of assertions, used to widen recall
Ollama	Runs the memory model and the embedding model locally
Qwen3-Embedding-0.6B [15]	1,024-number vectors for messages, assertions and questions
Fine-tuned Qwen3-4B-Instruct-2507	Extraction, consolidation and cited answers
PyTorch, Transformers, PEFT, bitsandbytes	QLoRA fine-tuning [14]
llama.cpp (GGUF, Q8_0)	Converts the trained model so Ollama can run it
Pi coding agent [9]	Sends messages to memory and receives recalled context
React, Vite; Docker Compose	Dashboard and graph view; runs all services

DATASET AND FINE-TUNING

Set	Details
Training data	2,961 synthetic rows in 16 domains, written and labelled by GPT-5.6-sol, cross-checked by a second model. Split 2,095 / 424 / 442.
Coverage set	480 casual two-person conversations; Claude Sonnet 5.5 labelled every fact (5,284 claims). Mixed with 600 older rows: 1,719 train, 670 validation.
Test data	LoCoMo [2]: 10 long conversations, 272 sessions, 1,540 questions. Never used for training.
Model	Qwen3-1.7B failed on real text (18 of 33 windows passed). Qwen3-4B with QLoRA in stages A, B, C. Stage C: 2.9 h on an 8 GB RTX 4070.

Training example

Assistant: “You could learn violin this winter.” User: “No, that is not my plan. I am learning watercolor painting.”

Expected memory: The user is learning watercolor painting. Only what the user said is stored.

DEMO (1/2): RECORDING AND ASKING IN A NEW SESSION

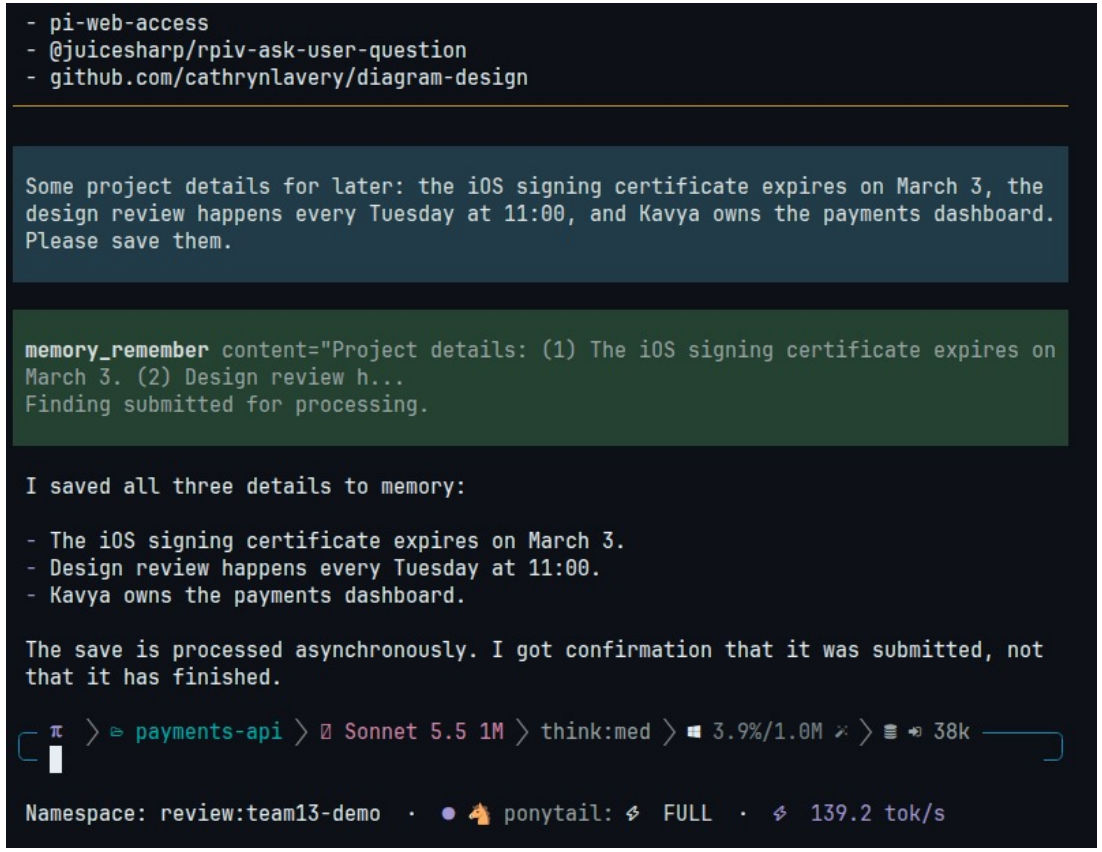


Fig. 5.1(a) Session 1: the user gives a certificate expiry date, a review time and a dashboard owner.

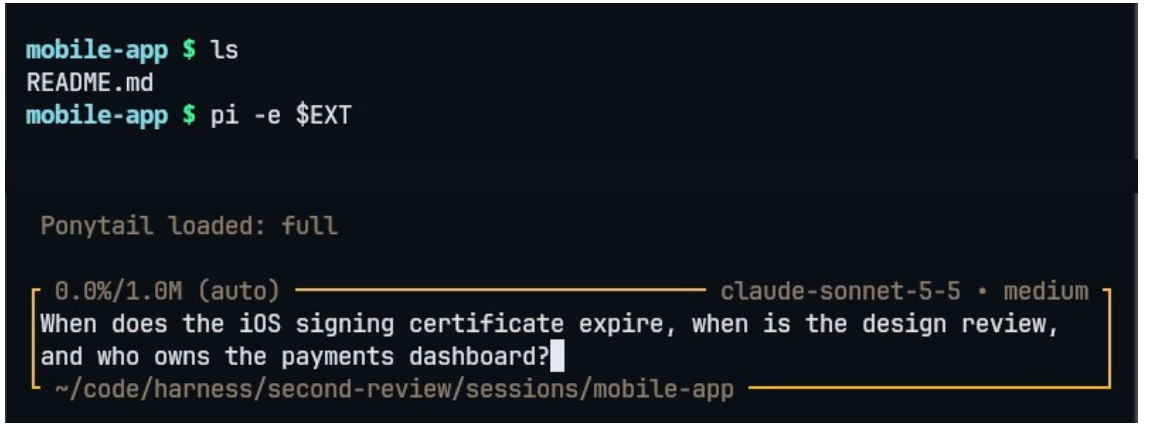


Fig. 5.1(b) A new session in another project asks for them without repeating them.

DEMO (2/2): RECALL AND CORRECTION

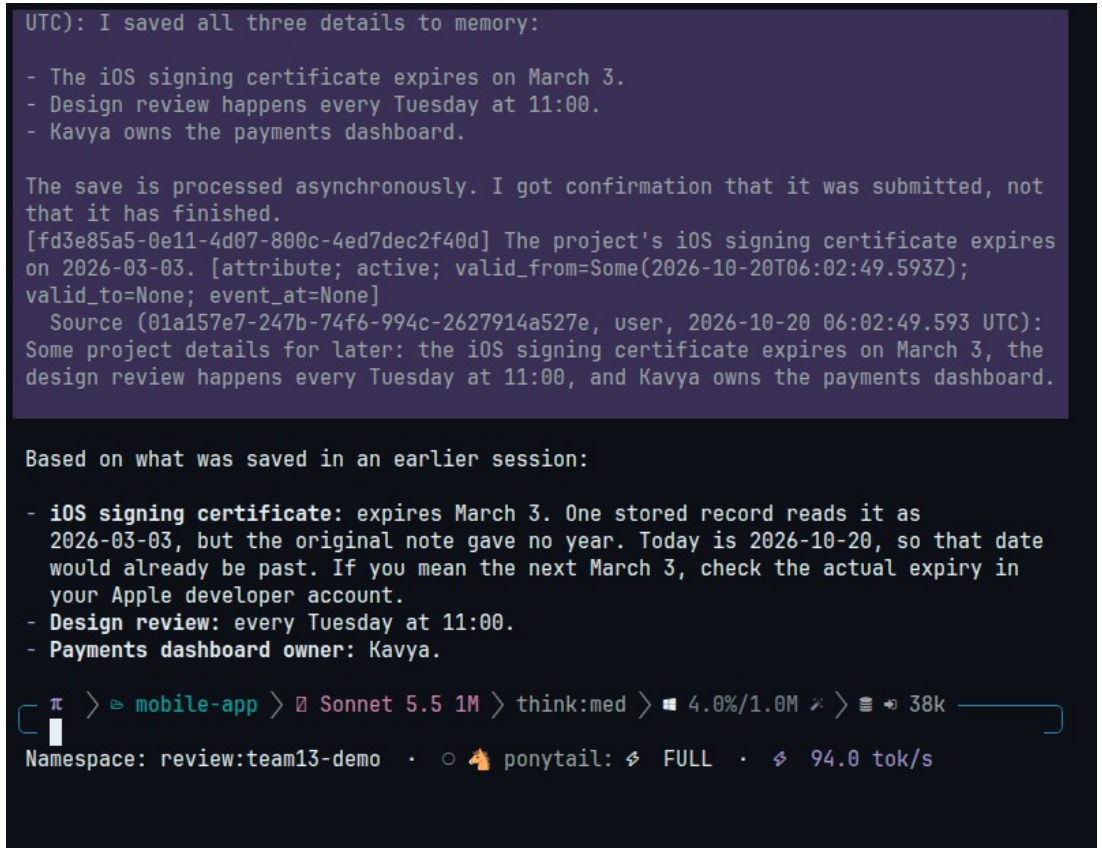


Fig. 5.2(a) Pi answers March 3, Tuesday 11:00 and Kavya from memory, and says where it came from.

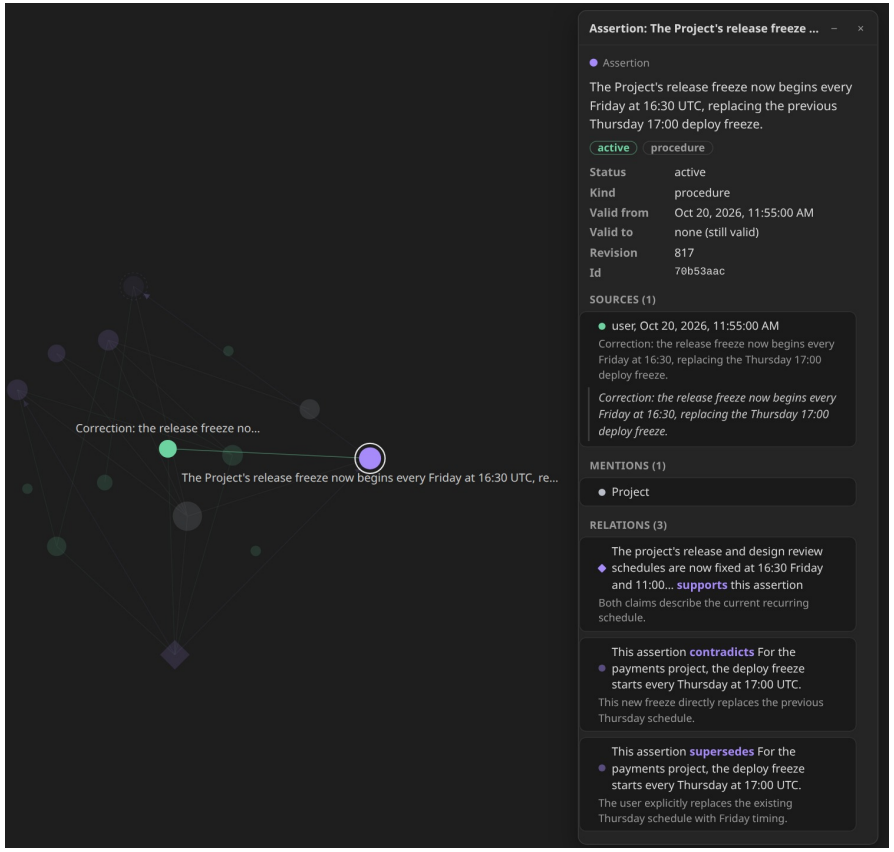


Fig. 5.2(b) The release freeze moved from Thursday 17:00 to Friday 16:30. The new value supersedes the old one.

RESULTS ON LOCOMO

Same reader (Claude Haiku 5.5) and prompt for every setup; only the context changes. Two blind judges: TypeSafe Jev [16] and GPT-4o-mini.

Context given to the reader	Accuracy (Jev)	GPT-4o-mini judge	Context tokens	Reader time
Memory engine	**83.1%**	**83.2%**	**6,790**	**5.2 s**
Plain retrieval (RAG)	80.0%	80.1%	4,473	5.5 s
Recent window (last 8,192 tokens)	43.0%	42.3%	8,168	5.2 s
Whole conversation	89.0%	88.8%	22,400	5.5 s
No memory	0.3%	0.4%	7	4.1 s

- The engine beats plain retrieval by 3.1 points (95% interval 1.2 to 4.8) and is 5.9 points below the whole conversation, using 30% of its tokens.
- The judges agree on 96.7% of answers (kappa 0.93). On the 8 conversations we never debugged on: 83.0% vs 79.4%.
- All 99,971 source quotes the engine returned match the conversation exactly. Recall takes 428 ms (median).

ANALYSIS AND COMPARISON

Category (Jev)	Engine	RAG	Whole
Single-hop	87.0	83.9	93.3
Multi-hop	83.0	74.5	88.7
Temporal	80.1	80.4	85.7
Open-domain	58.3	60.4	63.5

- Most of the gain is on multi-hop: one short assertion per person fits more facts in the budget.
- Ablations (300 questions): assertions alone 66.7%, without graph 76.3%, without consolidation 74.7%. Source text is still needed.
- Scripted tests: refused all 6 no-evidence questions, kept history in all 12 updates, never stored an API key.

System (own study)	Gap to full context
Mem0 with graph	−4.5
This work	**−5.6**
Mem0	−6.0
Zep	−6.9
LangMem [17]	−14.8
A-Mem [18]	−24.5

Published numbers are from the Mem0 paper [6] with a GPT-4o-mini reader. Our reader is stronger, so we compare each system's gap to the full context in its own study.

We are in the same range as Mem0 and Zep, not ahead.

CONCLUSION, LIMITATIONS AND FUTURE WORK

What we achieved

- All five modules work and run on one laptop, connected to Pi.
- 83.1% on LoCoMo, ahead of plain retrieval (80.0%) at 30% of the whole conversation's tokens.
- Corrections keep their history, and every answer traces back to an exact quote.

Challenges we hit

- Qwen3-1.7B was too small, so we moved to 4B.
- Ollama silently cut long inputs; we now check length and split.

Limitations

- One benchmark, one reader, model judges only.
- The extractor still misses facts (assertions alone: 66.7%).
- Graph and consolidation made no measurable difference on LoCoMo.
- LoCoMo numbers are from before the relative-date fix.

Future work

- Run LongMemEval and real coding sessions.
- Faster, more complete extraction (now 56 s per session).
- Access control, backups and human-checked judging.

REFERENCES (1/2)

[1] N. F. Liu et al., “Lost in the middle: How language models use long contexts,” Trans. Assoc. Comput. Linguist., vol. 12, pp. 157–173, 2024.

[2] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating very long-term conversational memory of LLM agents,” in Proc. 62nd Annu. Meeting Assoc. Comput. Linguist., 2024, pp. 13851–13870.

[3] C. Packer et al., “MemGPT: Towards LLMs as operating systems,” 2023, arXiv:2310.08560.

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, “Zep: A temporal knowledge graph architecture for agent memory,” 2025, arXiv:2501.13956.

[5] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, “LongMemEval: Benchmarking chat assistants on long-term interactive memory,” in Proc. ICLR, 2025, pp. 86809–86836.

[6] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, “Mem0: Building production-ready AI agents with scalable long-term memory,” in Proc. 28th ECAI, 2025, pp. 2993–3000.

[7] S. Daga, S. Sreedhar, and D. Shah, “LongMemEval,” Supermemory Research, May 2026. [Online]. Available: https://supermemory.ai/research/longmembench/ (accessed Oct. 6, 2026).

[8] C. Latimer et al., “Hindsight is 20/20: Building agent memory that retains, recalls, and reflects,” 2025, arXiv:2512.12818.

[9] M. Zechner et al., “pi: AI agent toolkit,” GitHub repository, 2026. [Online]. Available: https://github.com/earendil-works/pi (accessed Oct. 6, 2026).

REFERENCES (2/2)

[10] PostgreSQL Global Development Group, “Full text search,” PostgreSQL 17 Documentation, ch. 12, 2026. [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html (accessed Oct. 6, 2026).

[11] A. Kane et al., “pgvector: Open-source vector similarity search for Postgres,” GitHub repository, v0.8.6, 2026. [Online]. Available: https://github.com/pgvector/pgvector (accessed Oct. 6, 2026).

[12] Neo4j, Inc., “Query the database,” Neo4j HTTP API Manual, v5, 2026. [Online]. Available: https://neo4j.com/docs/http-api/5/query/ (accessed Oct. 6, 2026).

[13] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in Proc. 32nd ACM SIGIR, 2009, pp. 758–759.

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in Proc. NeurIPS, vol. 36, 2023, pp. 10088–10115.

[15] Y. Zhang et al., “Qwen3 Embedding: Advancing text embedding and reranking through foundation models,” 2025, arXiv:2506.05176.

[16] TypeSafe AI, “SystemOne API: the Jev decision model,” Web API, 2026. [Online]. Available: https://api.typesafe.ai/v1/systemone (accessed Oct. 21, 2026).

[17] LangChain, “LangMem SDK: Long-term memory for agents,” GitHub repository, 2025. [Online]. Available: https://github.com/langchain-ai/langmem (accessed Oct. 20, 2026).

[18] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-MEM: Agentic memory for LLM agents,” in Proc. NeurIPS, 2025, arXiv:2502.12110.